

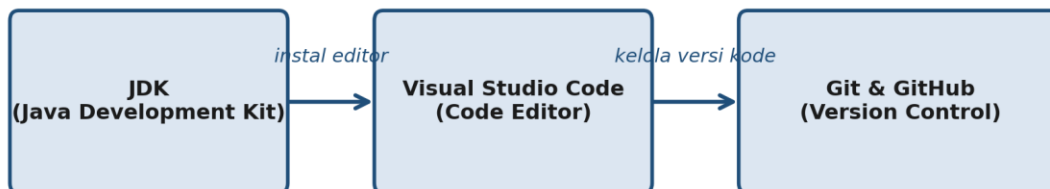
MODUL DASAR

Instalasi JDK & Visual Studio Code serta Tutorial Git

Dari Persiapan Lingkungan Pengembangan hingga Push Repository ke GitHub

Disusun sebagai bahan Ajar Praktikum

Alur Penyiapan Lingkungan Pengembangan



Daftar Isi

1. Instalasi Java Development Kit (JDK) di Windows 11
2. Instalasi Visual Studio Code di Windows 11
3. Menyiapkan VS Code untuk Pengembangan Java
4. Apa itu Git?
5. Instalasi Git di Windows 11
6. Konfigurasi Awal Git
7. Konsep Inti Git
8. Mengaktifkan Terminal di VS Code
9. Perintah Dasar Git (Praktik)
10. Branching
11. Menghubungkan Repository Lokal ke GitHub
12. Kolaborasi Lanjutan: Pull, Clone, Merge Conflict, dan .gitignore
13. Troubleshooting Umum
14. Rangkuman Perintah
15. Latihan Mandiri untuk Mahasiswa
16. Jobsheet Praktikum: Dari Berkas Lokal hingga Push ke GitHub (Publik)
17. Daftar Referensi

1. Instalasi Java Development Kit (JDK) di Windows 11

JDK (Java Development Kit) diperlukan untuk menulis, mengompilasi, dan menjalankan program Java. Versi yang direkomendasikan adalah versi LTS (Long-Term Support) terbaru, misalnya JDK 21 atau JDK 25.

Langkah	Keterangan
1	Kunjungi salah satu sumber unduhan resmi berikut: Oracle JDK (https://www.oracle.com/java/technologies/downloads/), OpenJDK (https://jdk.java.net/), atau Microsoft Build of OpenJDK (https://learn.microsoft.com/en-us/java/openjdk/download).
2	Pilih versi JDK LTS (misalnya JDK 21 atau JDK 25), lalu unduh installer Windows x64 (.exe).
3	Jalankan berkas .exe yang telah diunduh dengan klik dua kali, lalu ikuti wizard instalasi hingga selesai (pengaturan default sudah sesuai untuk kebanyakan pengguna).
4	Verifikasi instalasi dengan membuka Command Prompt (Win + R, ketik cmd, Enter), lalu jalankan perintah <code>java -version</code> dan <code>javac -version</code> .
5	(Opsional, disarankan) Atur environment variable <code>JAVA_HOME</code> agar mengarah ke folder instalasi JDK, lalu tambahkan <code>%JAVA_HOME%\bin</code> ke variabel Path sistem.

Catatan: Pengaturan `JAVA_HOME` dilakukan melalui menu "Environment Variables" pada System Properties Windows: klik New pada System Variables untuk `JAVA_HOME`, lalu edit variabel Path untuk menambahkan `%JAVA_HOME%\bin`.

2. Instalasi Visual Studio Code di Windows 11

Langkah	Keterangan
1	Kunjungi situs resmi https://code.visualstudio.com/
2	Klik tombol "Download for Windows" untuk mengunduh installer (User Installer 64-bit).
3	Jalankan berkas .exe yang telah diunduh.
4	Pada wizard instalasi, disarankan mencentang opsi: "Add to PATH", "Register Code as an editor for supported file types", dan "Add 'Open with Code' action" (agar dapat membuka folder/berkas langsung dari klik kanan).
5	Klik Install dan tunggu hingga proses selesai, lalu buka Visual Studio Code.

3. Menyiapkan VS Code untuk Pengembangan Java

Langkah	Keterangan
1	Buka VS Code, lalu buka tab Extensions (ikon kotak pada sidebar kiri, atau tekan Ctrl+Shift+X).
2	Cari ekstensi "Extension Pack for Java" (dari Microsoft) dan pasang (install). Paket ini sudah mencakup Language Support for Java, Debugger for Java, Test Runner for Java, Maven for Java, dan Project Manager for Java.
3	Setelah terpasang, mulai ulang (restart) VS Code.
4	Uji coba dengan membuat berkas baru bernama Hello.java, tulis program Java sederhana, lalu klik tombol "Run" yang muncul di atas method main.

Troubleshooting: Apabila muncul pesan "JDK not found" pada VS Code, buka File > Preferences > Settings, cari pengaturan java.jdt.ls.java.home, lalu isi dengan path lokasi instalasi JDK.

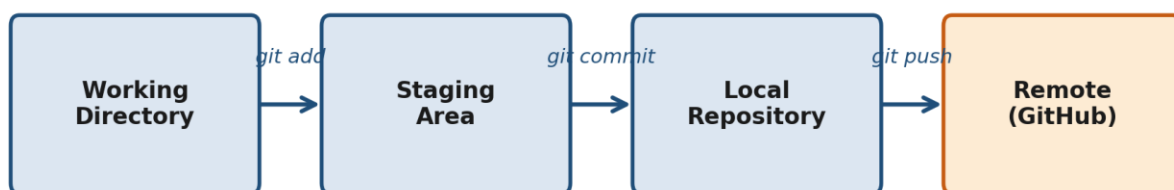
4. Apa itu Git?

Git adalah sistem kontrol versi (version control system) yang digunakan untuk melacak perubahan pada kode atau berkas dari waktu ke waktu. Dengan Git, sebuah tim dapat berkolaborasi dalam satu proyek, serta kembali ke versi sebelumnya apabila terjadi kesalahan.

Penting untuk membedakan dua istilah berikut:

- Git — perangkat lunak yang berjalan di komputer masing-masing pengguna.
- GitHub / GitLab / Bitbucket — layanan hosting daring (online) untuk menyimpan dan berbagi repository Git.

Alur Perubahan Berkas dalam Git



Gambar di atas menunjukkan alur perubahan berkas dalam Git: dari direktori kerja (working directory), masuk ke staging area melalui git add, disimpan sebagai commit di repository lokal, hingga akhirnya diunggah ke repository remote seperti GitHub melalui git push.

5. Instalasi Git di Windows 11

Langkah	Keterangan
1	Kunjungi situs resmi https://git-scm.com/downloads
2	Unduh installer untuk Windows, lalu jalankan berkas .exe tersebut.
3	Pada layar "Adjusting your PATH environment", pilih opsi "Git from the command line and also from 3rd-party software" agar Git dapat dipanggil dari terminal mana saja.
4	Lanjutkan proses instalasi dengan pengaturan default hingga selesai.
5	Verifikasi instalasi dengan membuka terminal (Command Prompt/PowerShell) dan menjalankan perintah git --version

Catatan: Jika terminal menampilkan pesan "git is not recognized..." setelah instalasi, tutup seluruh jendela VS Code/terminal lalu buka kembali. Perubahan PATH baru terbaca setelah terminal dibuka ulang.

6. Konfigurasi Awal Git

Sebelum membuat commit pertama, identitas pengguna wajib diatur terlebih dahulu:

```
git config --global user.name "Nama Lengkap"
git config --global user.email "email@contoh.com"
```

Untuk memeriksa atau mengubah email/nama yang sudah tersimpan:

```
git config --global user.email
git config --global user.email "email-baru@contoh.com"
```

7. Konsep Inti Git

Perintah	Fungsi
Repository (repo)	Folder proyek yang dilacak perubahannya oleh Git.
Commit	"Snapshot" atau catatan perubahan yang disimpan beserta pesan penjelasan.
Branch	Jalur pengembangan terpisah, misalnya untuk fitur atau eksperimen baru.
Working directory	Berkas-berkas yang sedang diedit oleh pengguna.
Staging area	Area "persiapan" berisi perubahan yang akan disertakan pada commit berikutnya.
Remote	Repository yang disimpan di server, misalnya GitHub.

8. Mengaktifkan Terminal di VS Code

Terminal dapat diaktifkan melalui salah satu cara berikut:

- Shortcut keyboard: Ctrl + ` (tombol backtick)
- Menu: Terminal > New Terminal
- Command Palette (Ctrl+Shift+P), lalu ketik "Terminal: Create New Terminal"

Disarankan membuka folder proyek terlebih dahulu (File > Open Folder) sebelum membuka terminal, agar terminal langsung aktif pada lokasi folder proyek tersebut.

9. Perintah Dasar Git (Praktik)

9.1 Membuat Repository Baru

```
mkdir belajar-git  
cd belajar-git  
git init
```

Perintah git init menandai folder tersebut sebagai repository Git yang baru dan kosong.

9.2 Membuat dan Melacak Berkas

```
echo "Belajar Git hari ini" > catatan.txt  
git status
```

Berkas yang baru dibuat akan berstatus "Untracked" — Git mengetahui keberadaannya, namun belum melacak perubahannya.

9.3 Staging dan Commit

```
git add catatan.txt  
git commit -m "Membuat file catatan pertama"  
git log
```

git add memindahkan berkas ke staging area, sedangkan git commit menyimpan snapshot perubahan secara permanen ke riwayat proyek.

9.4 Alur Kerja Berulang: Edit → Add → Commit

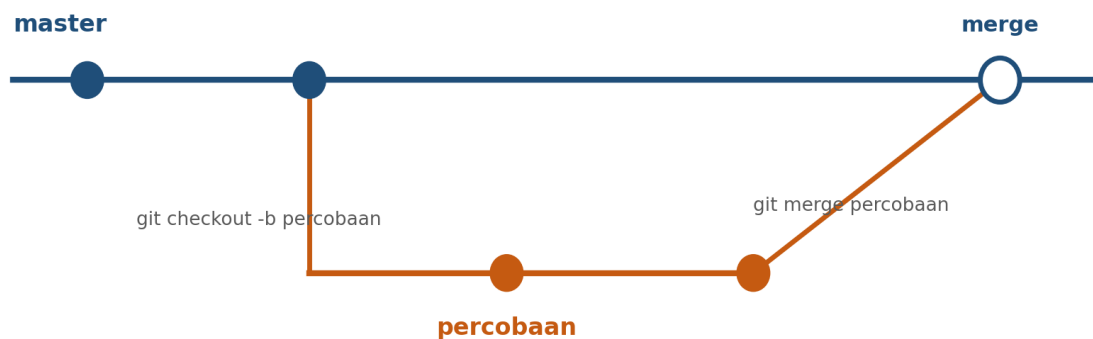
Langkah	Keterangan
1	Edit berkas yang sudah ada (misalnya menambah baris baru pada catatan.txt).
2	Jalankan git status — berkas akan berstatus "modified".
3	Jalankan git diff untuk melihat detail baris yang berubah (opsional).
4	Jalankan git add nama_berkas untuk staging.
5	Jalankan git commit -m "pesan perubahan" untuk menyimpan snapshot baru.

10. Branching

Branch memungkinkan pengembangan fitur atau eksperimen baru dilakukan tanpa mengganggu kode utama (biasanya branch master atau main). Apabila hasil pengembangan dinilai baik, branch tersebut dapat digabungkan (merge).

Perintah	Fungsi
<code>git branch</code>	Menampilkan daftar branch dan menandai branch aktif.
<code>git branch nama-branch</code>	Membuat branch baru tanpa berpindah ke branch tersebut.
<code>git checkout nama-branch</code>	Berpindah ke branch yang dituju.
<code>git checkout -b nama-branch</code>	Membuat sekaligus berpindah ke branch baru.
<code>git merge nama-branch</code>	Menggabungkan branch yang disebut ke branch yang sedang aktif.
<code>git log --all --oneline --graph</code>	Menampilkan visualisasi percabangan riwayat commit.

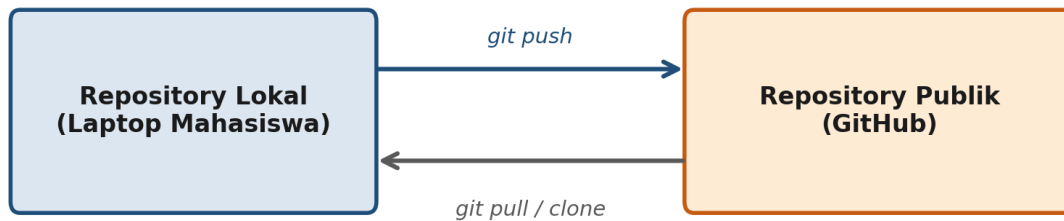
Ilustrasi Branching dan Merge



Latihan: Buat branch baru bernama "percobaan", lakukan perubahan dan commit di branch tersebut, lalu kembali ke branch master dan amati bahwa perubahan belum muncul sebelum dilakukan git merge.

11. Menghubungkan Repository Lokal ke GitHub

Sinkronisasi Repository Lokal ke GitHub



Langkah	Keterangan
1	Buat akun pada https://github.com apabila belum memiliki akun.
2	Di GitHub, klik tombol "+" pada pojok kanan atas, lalu pilih "New repository". Beri nama repository, dan jangan mencentang opsi "Add a README" karena proyek sudah memiliki isi dari lokal.
3	Salin URL repository yang diberikan GitHub, contoh: https://github.com/username/nama-repo.git
4	Pada terminal, hubungkan repository lokal dengan remote GitHub menggunakan git remote add origin, lalu unggah riwayat commit menggunakan git push.
5	Saat diminta autentikasi, ikuti proses login melalui browser (atau gunakan Personal Access Token bila diminta username/password langsung di terminal).
<pre>git checkout master git remote add origin https://github.com/username/nama-repo.git git push -u origin master</pre>	

Penting: Sejak tahun 2021 GitHub tidak lagi menerima password akun biasa untuk operasi Git melalui terminal. Sebagai gantinya, gunakan Personal Access Token (PAT) yang dibuat melalui menu **Settings > Developer settings > Personal access tokens**, atau login melalui jendela browser yang muncul otomatis (Git Credential Manager).

12. Kolaborasi Lanjutan: Pull, Clone, Merge Conflict, dan .gitignore

Setelah menguasai alur dasar (init, add, commit, branch, push), langkah selanjutnya yang natural untuk dipelajari adalah bekerja secara kolaboratif — baik sendiri di beberapa perangkat, maupun bersama anggota tim lain.

12.1 git pull — Mengambil Perubahan Terbaru dari GitHub

git pull digunakan untuk mengambil (download) dan menggabungkan perubahan terbaru dari repository remote (misalnya GitHub) ke repository lokal. Perintah ini penting ketika bekerja di banyak perangkat, atau ketika berkolaborasi dengan anggota tim lain yang juga melakukan push ke repository yang sama.

```
git pull origin master
```

Praktik yang baik adalah selalu menjalankan git pull sebelum mulai bekerja, agar perubahan yang dimiliki selalu sesuai dengan versi terbaru di remote.

12.2 git clone — Menyalin Repository dari GitHub

git clone digunakan untuk menyalin (download) seluruh isi dan riwayat commit sebuah repository dari GitHub ke komputer lokal, sekaligus otomatis menghubungkannya sebagai remote bernama origin. Perintah ini biasa digunakan saat mulai bekerja pada proyek yang sudah ada di GitHub, misalnya di laptop atau komputer lain.

```
git clone https://github.com/username/nama-repo.git
```

Setelah proses clone selesai, sebuah folder baru dengan nama repository akan terbentuk, lengkap dengan seluruh riwayat commit yang bisa langsung dilanjutkan pengerjaannya.

12.3 Menangani Merge Conflict

Merge conflict terjadi ketika dua perubahan yang berbeda dilakukan pada baris yang sama di sebuah berkas, sehingga Git tidak dapat menggabungkannya secara otomatis — baik saat git merge maupun git pull.

Saat konflik terjadi, Git akan menandai bagian yang bentrok pada berkas menggunakan penanda berikut:

```
<<<<<< HEAD
isi perubahan pada branch aktif
=====
isi perubahan dari branch/remote yang digabungkan
>>>>>> nama-branch
```

Langkah	Keterangan
1	Jalankan git status untuk melihat berkas mana saja yang mengalami konflik.
2	Buka berkas tersebut, lalu tentukan perubahan mana yang akan dipertahankan (bisa salah satu, gabungan keduanya, atau versi baru hasil penyesuaian).

3	Hapus seluruh penanda konflik (<<<<<<<, =====, >>>>>>) setelah isi berkas diputuskan.
4	Simpan berkas, lalu jalankan git add pada berkas yang telah diperbaiki untuk menandai konflik telah selesai.
5	Jalankan git commit untuk menyelesaikan proses merge (pesan commit biasanya sudah terisi otomatis oleh Git).
Tips: Semakin sering melakukan commit kecil dan git pull secara berkala, semakin kecil kemungkinan terjadi merge conflict yang besar dan rumit.	

12.4 .gitignore — Mengabaikan Berkas yang Tidak Perlu Dilacak

.gitignore adalah berkas khusus yang memberi tahu Git berkas atau folder mana saja yang tidak perlu dilacak perubahannya. Ini sangat penting terutama saat mulai mengerjakan proyek nyata, karena tidak semua berkas dalam sebuah proyek perlu (atau boleh) diunggah ke repository, misalnya:

- Hasil kompilasi/build, seperti folder bin/, build/, atau target/
- Dependensi pihak ketiga yang berukuran besar, seperti folder node_modules/
- Berkas konfigurasi lokal yang berisi data sensitif, seperti .env
- Berkas sementara sistem operasi atau editor, seperti .DS_Store atau .vscode/

Cara membuatnya: buat berkas baru bernama persis .gitignore pada folder utama proyek (root), lalu tuliskan pola nama berkas/folder yang ingin diabaikan, satu pola per baris:

```
bin/
build/
node_modules/
*.log
.env
```

Berkas atau folder yang sudah tercantum di .gitignore tidak akan muncul lagi sebagai "untracked" pada git status, dan tidak akan ikut ter-commit maupun ter-push ke GitHub.

Catatan: Apabila sebuah berkas sudah pernah di-commit sebelum ditambahkan ke .gitignore, Git akan tetap melacaknya. Berkas tersebut perlu dihapus dari pelacakan terlebih dahulu menggunakan `git rm --cached nama_berkas`, baru kemudian aturan .gitignore akan berlaku untuknya.

13. Troubleshooting Umum

13.1 Pesan "git is not recognized"

Penyebab: Git belum terinstal, atau sudah terinstal namun PATH belum terbaca oleh terminal yang sedang berjalan.

Solusi: Periksa keberadaan folder instalasi Git (umumnya di C:\Program Files\Git). Jika sudah ada, tutup seluruh jendela VS Code/terminal dan buka kembali. Jika folder tidak ditemukan, ulangi proses instalasi dan pastikan opsi PATH dicentang.

13.2 Nama Berkas Tidak Sesuai (contoh: koma menggantikan titik)

Kesalahan pengetikan nama berkas, misalnya catatan,txt alih-alih catatan.txt, akan membuat Git menganggapnya sebagai berkas baru yang terpisah. Periksa nama berkas menggunakan perintah `dir` (Command Prompt) atau `ls` (Git Bash), lalu perbaiki dengan mengganti nama (`ren`) atau menghapus (`del`) berkas yang salah.

```
dir
ren catatan,txt catatan.txt
del catatan,txt
```

13.3 Commit dengan Identitas Penulis yang Salah

Apabila commit terakhir menggunakan nama/email yang belum diperbarui dan belum pernah diunggah (push) ke remote, dapat diperbaiki dengan perintah berikut:

```
git commit --amend --author="Nama <email@contoh.com>" --no-edit
```

Perhatian: Perintah `--amend` mengubah riwayat commit (menghasilkan ID/hash baru). Aman digunakan selama commit tersebut belum pernah di-push ke remote.

14. Rangkuman Perintah

Perintah	Fungsi
<code>git init</code>	Membuat repository Git baru pada folder aktif.
<code>git status</code>	Menampilkan status perubahan berkas.
<code>git add nama_berkas</code>	Menambahkan berkas ke staging area.
<code>git commit -m "pesan"</code>	Menyimpan snapshot perubahan (commit).
<code>git log</code>	Menampilkan riwayat commit.
<code>git diff</code>	Menampilkan detail perubahan yang belum di-stage.
<code>git branch</code>	Menampilkan/membuat branch.
<code>git checkout</code>	Berpindah antar branch.
<code>git merge</code>	Menggabungkan branch.
<code>git remote add origin <url></code>	Menghubungkan repository lokal ke remote.
<code>git push</code>	Mengunggah commit ke remote.
<code>git pull</code>	Mengambil perubahan terbaru dari remote.
<code>git clone <url></code>	Menyalin repository dari remote ke lokal.
<code>git rm --cached nama_berkas</code>	Menghentikan pelacakan berkas yang sudah pernah di-commit.
<code>.gitignore</code>	Berkas konfigurasi berisi daftar berkas/folder yang tidak dilacak Git.

15. Latihan Mandiri untuk Mahasiswa

Instruksikan mahasiswa untuk menyelesaikan latihan berikut secara mandiri sebagai bentuk evaluasi pemahaman:

- Instal Git dan verifikasi dengan **git --version**.
- Buat repository baru bernama "**praktikum-git**" dan lakukan minimal tiga kali commit dengan pesan yang jelas.
- Buat branch baru bernama "**fitur-baru**", lakukan perubahan, lalu gabungkan (merge) kembali ke branch utama.
- Buat repository pada GitHub, hubungkan dengan repository lokal, dan unggah (push) seluruh riwayat commit.
- Ambil tangkapan layar (screenshot) dari halaman repository di GitHub sebagai bukti pengumpulan tugas.
- Buat berkas .gitignore pada repository, tambahkan minimal satu folder dan satu pola berkas yang diabaikan, lalu buktikan bahwa berkas tersebut tidak ikut ter-commit.
- Simulasikan penggunaan git clone dengan menyalin repository yang sudah dibuat ke folder lain, lalu lakukan perubahan dan git pull untuk menyinkronkannya.
- Buat simulasi merge conflict dengan mengubah baris yang sama pada dua branch berbeda, lalu selesaikan konfliknya secara manual.

16. Jobsheet Praktikum: Dari Berkas Lokal hingga Push ke GitHub (Publik)

A. Tujuan Praktikum

- Mahasiswa mampu membuat dan mengelola berkas proyek pada direktori lokal.
- Mahasiswa mampu menginisialisasi repository Git dan melakukan pelacakan perubahan berkas.
- Mahasiswa mampu menggunakan perintah dasar Git (add, commit, branch, merge) dengan benar.
- Mahasiswa mampu menghubungkan repository lokal dengan repository publik di GitHub dan mengunggah (push) hasil pekerjaan.

B. Alat dan Bahan

- Laptop dengan sistem operasi Windows 11.
- Git (sudah terinstal dan terverifikasi dengan `git --version`).
- Visual Studio Code (sudah terinstal).
- Akun GitHub aktif.
- Koneksi internet.

C. Langkah Kerja

Tahap 1 — Menyiapkan Direktori Proyek Lokal

Langkah	Keterangan
1	Buka VS Code, lalu aktifkan terminal (Ctrl + `).
2	Buat folder proyek baru dengan nama sesuai NIM/nama, misalnya jobsheet-git-nim, menggunakan perintah <code>mkdir jobsheet-git-nim</code> lalu masuk ke folder tersebut dengan <code>cd jobsheet-git-nim</code> .
3	Buka folder tersebut di VS Code melalui File > Open Folder agar seluruh proses berikutnya dilakukan pada direktori yang benar.
4	Buat berkas baru bernama <code>profil.txt</code> yang berisi identitas singkat (nama, NIM, kelas).

Tahap 2 — Inisialisasi Repository dan Commit Pertama

Langkah	Keterangan
5	Jalankan git init untuk menjadikan folder tersebut sebagai repository Git.
6	Atur identitas Git apabila belum diatur sebelumnya, menggunakan git config --global user.name dan git config --global user.email.
7	Jalankan git status untuk memastikan profil.txt berstatus "Untracked".
8	Jalankan git add profil.txt untuk memasukkan berkas ke staging area.
9	Jalankan git commit -m "Inisialisasi proyek dan profil mahasiswa" untuk membuat commit pertama.

Tahap 3 — Menambah Berkas dan Riwayat Perubahan

Langkah	Keterangan
10	Buat berkas kedua bernama tugas.txt berisi deskripsi singkat tugas yang sedang dikerjakan.
11	Jalankan git add tugas.txt lalu git commit -m "Menambahkan berkas tugas".
12	Edit kembali profil.txt (tambahkan satu baris informasi baru), simpan, lalu jalankan git status dan git diff untuk mengamati perubahan sebelum di-commit.
13	Lakukan git add dan git commit untuk perubahan tersebut dengan pesan commit yang jelas.
14	Jalankan git log --oneline untuk memastikan terdapat minimal tiga commit.

Tahap 4 — Branching dan Merge

Langkah	Keterangan
15	Buat branch baru bernama pengembangan menggunakan git checkout -b pengembangan.
16	Pada branch pengembangan, buat berkas baru bernama catatan-pengembangan.txt, isi bebas, lalu lakukan add dan commit.
17	Kembali ke branch utama (master atau main) menggunakan git checkout master, lalu amati bahwa berkas catatan-pengembangan.txt belum muncul di branch ini.
18	Gabungkan branch pengembangan ke branch utama menggunakan git merge pengembangan.
19	Verifikasi hasil penggabungan dengan git log --all --oneline --graph.

Tahap 5 — Membuat Repository Publik di GitHub

Langkah	Keterangan
20	Login ke akun GitHub, klik tombol "+" pada pojok kanan atas, lalu pilih "New repository".
21	Beri nama repository sesuai folder proyek (misalnya jobsheet-git-nim), pastikan visibilitas diatur ke "Public", dan jangan mencentang opsi "Add a README".
22	Klik "Create repository", lalu salin URL repository yang tersedia (format https://github.com/username/nama-repo.git).

Tahap 6 — Menghubungkan dan Push ke GitHub

Langkah	Keterangan
23	Di terminal VS Code, pastikan berada pada branch utama menggunakan git checkout master.
24	Hubungkan repository lokal dengan repository GitHub menggunakan git remote add origin <url-repository>.
25	Unggah seluruh riwayat commit menggunakan git push -u origin master.
26	Apabila diminta autentikasi, login melalui jendela browser yang muncul otomatis, atau gunakan Personal Access Token apabila diminta username dan password secara langsung.
27	Setelah proses push selesai, buka halaman repository pada browser dan refresh untuk memastikan seluruh berkas dan riwayat commit telah tampil.

Tahap 7 — Menerapkan .gitignore

Langkah	Keterangan
28	Buat folder baru bernama build/ di dalam proyek, dan buat sebuah berkas contoh di dalamnya (misalnya build/output.log).
29	Buat berkas baru bernama .gitignore pada folder utama proyek, lalu tuliskan build/ di dalamnya.
30	Jalankan git status dan amati bahwa folder build/ tidak lagi muncul sebagai berkas yang belum dilacak (untracked).
31	Lakukan git add .gitignore dan git commit -m "Menambahkan .gitignore" untuk menyimpan aturan tersebut ke repository.

Tahap 8 — Simulasi Merge Conflict

Langkah	Keterangan
32	Pada branch master, edit baris pertama profil.txt, lalu add dan commit dengan pesan "Mengubah profil dari master".
33	Pindah ke branch pengembangan menggunakan git checkout pengembangan, edit baris yang sama pada profil.txt dengan isi berbeda, lalu add dan commit.
34	Kembali ke branch master, lalu jalankan git merge pengembangan — Git akan menampilkan pesan konflik (conflict) pada profil.txt.
35	Buka profil.txt, amati penanda konflik (<<<<<<, =====, >>>>>>), tentukan isi akhir yang benar, lalu hapus seluruh penanda tersebut.
36	Simpan berkas, jalankan git add profil.txt, lalu git commit untuk menyelesaikan proses merge.

Tahap 9 — Simulasi Clone dan Pull (Kolaborasi Antar Perangkat)

Langkah	Keterangan
37	Buat folder terpisah di lokasi lain pada laptop (mensimulasikan perangkat kedua), lalu jalankan git clone <url-repository> di dalam folder tersebut.
38	Pada repository asli (folder pertama), buat perubahan baru pada salah satu berkas, lalu add, commit, dan push ke GitHub.
39	Pada repository hasil clone (folder kedua), jalankan git pull origin master, lalu amati bahwa perubahan dari repository asli sudah masuk.

D. Bukti Pengerjaan yang Dikumpulkan

- Tangkapan layar (screenshot) hasil **git log --oneline** yang menunjukkan minimal tiga commit.
- Tangkapan layar hasil **git log --all --oneline --graph** yang menunjukkan percabangan dan penggabungan branch.
- Tautan (link) repository publik di GitHub yang sudah berisi seluruh berkas dan riwayat commit.
- Tangkapan layar halaman repository di GitHub yang menampilkan daftar berkas (profil.txt, tugas.txt, catatan-pengembangan.txt).
- Tangkapan layar isi berkas .gitignore beserta hasil git status yang menunjukkan folder build/ tidak dilacak.
- Tangkapan layar proses penyelesaian merge conflict pada profil.txt, sebelum dan sesudah penanda konflik dihapus.
- Tangkapan layar hasil git clone pada folder simulasi perangkat kedua, serta hasil git pull yang menunjukkan sinkronisasi berhasil.

E. Rubrik Penilaian

Perintah	Fungsi
Inisialisasi & commit lokal	Repository berhasil dibuat, minimal tiga commit dengan pesan yang jelas dan deskriptif.
Branching & merge	Branch pengembangan berhasil dibuat, berisi commit tersendiri, dan berhasil digabungkan ke branch utama tanpa konflik.
Repository GitHub	Repository publik berhasil dibuat dan dapat diakses melalui tautan yang diberikan.
Push & sinkronisasi	Seluruh riwayat commit lokal berhasil terunggah dan tampil sesuai di GitHub.
.gitignore	Berkas .gitignore diterapkan dengan benar dan folder/berkas yang dituju terbukti tidak ikut dilacak.
Penanganan merge conflict	Konflik berhasil diselesaikan secara manual dengan isi akhir yang tepat dan penanda konflik terhapus seluruhnya.
Clone & pull	Simulasi clone dan pull berhasil menyinkronkan perubahan antara dua salinan repository.
Kerapian dokumentasi	Pesan commit informatif dan berkas tersusun rapi sesuai instruksi jobsheet.

17. Daftar Referensi

1. Git. (2024). Git — Reference Manual. Diakses dari <https://git-scm.com/doc>
2. Git. (2024). Git — Downloads. Diakses dari <https://git-scm.com/downloads>
3. GitHub, Inc. (2024). GitHub Docs — Getting started with Git. Diakses dari <https://docs.github.com/en/get-started>
4. GitHub, Inc. (2024). GitHub Docs — Managing remote repositories. Diakses dari <https://docs.github.com/en/repositories>
5. GitHub, Inc. (2024). GitHub Docs — Authenticating with a personal access token. Diakses dari <https://docs.github.com/en/authentication>
6. Microsoft. (2024). Visual Studio Code Docs. Diakses dari <https://code.visualstudio.com/docs>
7. Microsoft. (2024). Visual Studio Code — Java in VS Code. Diakses dari <https://code.visualstudio.com/docs/languages/java>
8. Oracle Corporation. (2024). Java SE Downloads and Documentation. Diakses dari <https://www.oracle.com/java/technologies/downloads/>
9. Eclipse Adoptium / OpenJDK. (2024). Eclipse Temurin — JDK Downloads. Diakses dari <https://jdk.java.net/>
10. Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Tersedia daring di <https://git-scm.com/book>